Universidad de Cundinamarca
Facultad de Ingeniería
Programa de Ingeniería de Sistemas y Computación
CADI Redes y Comunicación

Taller Redes Masivas de Servidores

1. Diseño de la interfaz

Crear una página web titulada Panel de administración de servidores dinámicos, que permita visualizar la información del sistema mediante una interfaz organizada, intuitiva y funcional.

La página debe contener las siguientes secciones:

1. **Encabezado:** nombre del sistema y descripción breve de su funcionamiento.
2. **Resumen del sistema:** tarjetas informativas con el total de solicitudes atendidas y el número de servidores configurados.
3. **Listado de servidores:** tabla con el nombre, puerto y estado de cada servidor.
4. **Gestión de solicitudes:** botón para enviar una solicitud al balanceador y visualizar el resultado.
5. **Estadísticas:** sección para mostrar la cantidad de solicitudes procesadas por cada servidor.
6. **Mensajes del sistema:** área para informar resultados, errores o problemas de conexión.

2. Conexión con la API

Utilizar JavaScript y el método fetch() para consumir los endpoints desarrollados en clase.

1. Consultar los servidores

Consumir el endpoint:
GET /servidores

- Obtener los datos de los servidores registrados.
- Mostrar el nombre, el puerto y el estado de cada servidor.
- Diferenciar visualmente los servidores activos de los inactivos.
- Crear un botón para actualizar la información.

2. Enviar solicitudes

Consumir el endpoint:
GET /solicitar

- Crear un botón denominado **Enviar solicitud**.
- Enviar una petición al balanceador cuando se haga clic.
- Mostrar el servidor que atendió la solicitud.
- Presentar el resultado devuelto por el servidor.
- Informar al usuario si la solicitud no pudo procesarse.

3. Consultar las estadísticas

Consumir el endpoint:
GET /estadisticas

- Mostrar el total de solicitudes atendidas.
- Presentar la cantidad de solicitudes procesadas por cada servidor.
- Representar los resultados mediante tarjetas, tablas o barras de progreso.
- Actualizar las estadísticas después de enviar una solicitud.

4. Manejar errores

Implementar mecanismos para informar cuando:

- La API no responde.
- No hay servidores disponibles.
- Ocurre un error durante una petición.
- No se pueden consultar las estadísticas.

Los mensajes deben visualizarse directamente en la página y ser comprensibles para el usuario.

3. Requisitos de desarrollo

- Utilizar HTML para estructurar la interfaz.
- Utilizar CSS para el diseño y la presentación visual.
- Utilizar JavaScript para consumir la API.
- Procesar las respuestas JSON de los endpoints.
- Obtener los datos directamente de la API, sin utilizar información inventada.
- Implementar todos los botones y verificar su funcionamiento.
- Mantener el código organizado y legible.
- No modificar ni reconstruir la API desarrollada en clase.

4. Pruebas funcionales

Realizar las siguientes pruebas con la API en ejecución:

1. Consultar y visualizar los servidores disponibles.
2. Enviar cinco solicitudes y observar qué servidor atiende cada una.
3. Actualizar la información después de detener uno de los servidores.
4. Verificar que las estadísticas cambien al enviar nuevas solicitudes.
5. Comprobar el mensaje que aparece cuando no hay servidores disponibles.
6. Verificar que la interfaz pueda actualizar la información cuando los servidores vuelvan a funcionar.

Registrar evidencias de las pruebas realizadas.

5. Entregables

Cada grupo debe presentar:

- Archivos fuente index.html, estilos.css y script.js.
- Interfaz web funcional conectada a la API.
- Evidencias de ejecución y pruebas.

- Demostración en vivo.
- Informe final